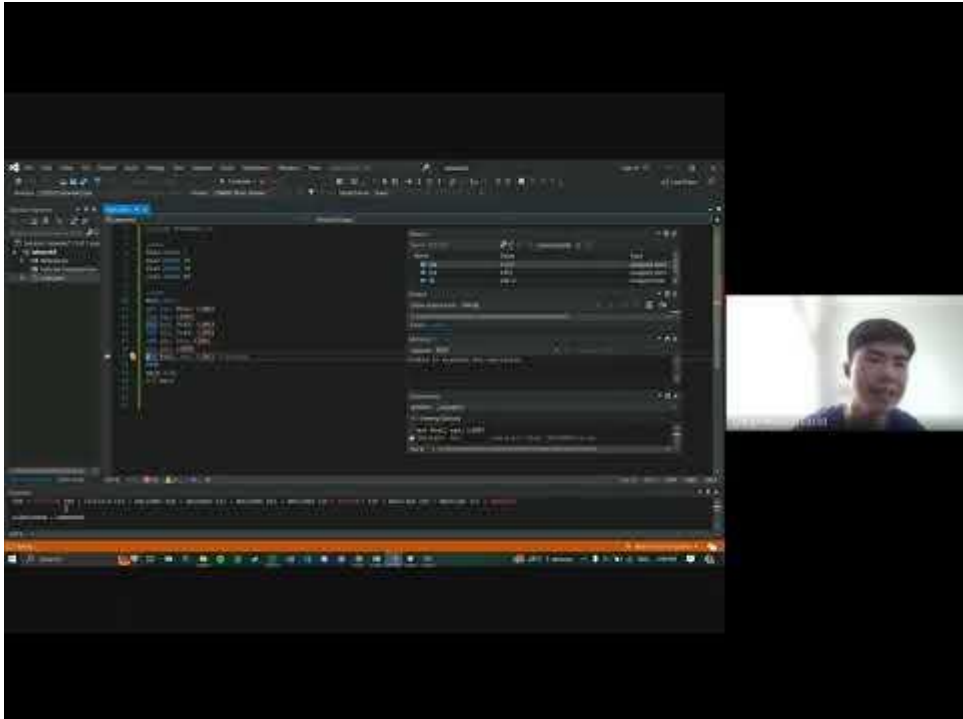




Department of Computer Science
Faculty of Computing
UNIVERSITI TEKNOLOGI MALAYSIA

SUBJECT NAME:	COMPUTER ORGANIZATION AND ARCHITECTURE				
SUBJECT CODE:	SECR 1033				
SEMESTER:	2 – 2023/2024				
LAB TITLE:	Lab 2: Arithmetic Equations & Operations				
STUDENT INFO :	Execute the lab in groups of two. <table border="1"> <thead> <tr> <th>Student 1</th><th>Student 2</th></tr> </thead> <tbody> <tr> <td>No. 1, 3, 5</td><td>No 2, 4, 6</td></tr> </tbody> </table> Name 1: Damiya Aina binti Basir Abd Shammad Metric No: A23CS0220 Name 2: Lim Jia An Metric No: A22EE0110	Student 1	Student 2	No. 1, 3, 5	No 2, 4, 6
Student 1	Student 2				
No. 1, 3, 5	No 2, 4, 6				

	Youtube link : 
SUBMISSION DATE:	22/05/2024

MARKS:

Arithmetic Equation Coding in Assembly Language

Q1. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 1 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

```
INCLUDE Irvine32.inc
.data
var1 word 1
var2 word 9

.code
main PROC
    mov ax, var1      ; LINE1
    mov bx, var2      ; LINE2
    xchg ax, bx       ; LINE3
    mov var1, ax      ; LINE4
    mov var2, bx      ; LINE5
    call DumpRegs
    exit
main ENDP
END main
```

Answer Q1

a) Fill (Write) in the contents for the related register in each line:

Table 1

LINE1	AX = 0001h var1 = 0001h	Move the value of var1 (1d) into register AX
LINE2	BX = 0009h var2 = 0009h	Move the value of var2 (9d) into register BX.
LINE3	AX = 0009h BX = 0001h	Register BX exchange with register AX with 16-bits register
LINE4	AX = 0009h var1 = 0009h	Move register AX into value of var1 (1d)
LINE5	BX = 0001h var2 = 0001h	Move register BX into value of var2 (9d)

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

```
EAX = 001E0001 EBX = 002DA000 ECX = 00A710AA EDX = 00A710AA ESI = 00A710AA EDI = 00A710AA
EIP = 00A73666 ESP = 001EFE34 EBP = 001EFE40 EFL = 00000246
```

LINE2:

```
EAX = 001E0001 EBX = 002D0009 ECX = 00A710AA EDX = 00A710AA ESI = 00A710AA EDI = 00A710AA
EIP = 00A7366D ESP = 001EFE34 EBP = 001EFE40 EFL = 00000246
```

LINE3:

```
EAX = 001E0009 EBX = 002D0001 ECX = 00A710AA EDX = 00A710AA ESI = 00A710AA EDI = 00A710AA
EIP = 00A7366F ESP = 001EFE34 EBP = 001EFE40 EFL = 00000246
```

LINE4:

```
EAX = 001E0009 EBX = 002D0001 ECX = 00A710AA EDX = 00A710AA ESI = 00A710AA EDI = 00A710AA  
EIP = 00A73675 ESP = 001EFE34 EBP = 001EFE40 EFL = 00000246
```

LINE5:

```
EAX = 001E0009 EBX = 002D0001 ECX = 00A710AA EDX = 00A710AA ESI = 00A710AA EDI = 00A710AA  
EIP = 00A7367C ESP = 001EFE34 EBP = 001EFE40 EFL = 00000246
```

Q2. Execute the program below. Determine output of the program by inspecting the content of the related registers and watches.

a) Fill in Table 2 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $Rval = (-Xval + (Yval - Zval)) + 1$

```
include irvine32.inc

.data
Rval DWORD ?
Xval DWORD 26
Yval DWORD 30
Zval DWORD 40

.code
main proc
    mov eax,Xval      ; LINE1
    neg eax           ; LINE2
    mov ebx,Yval      ; LINE3
    sub ebx,Zval      ; LINE4
    add eax,ebx       ; LINE5
    inc eax           ; LINE6
    mov Rval,eax      ; LINE7
    exit
main endp
end main
```

Answer Q2

a) Fill (Write) in the contents for the related register in each line:

Table 2

LINE1	EAX = 0000001Ah Xval = 0000001Ah	Move the value of Xval (26d) into register EAX
LINE2	EAX = FFFFFFFE6	Negate the value of register EAX from 26d to -26d
LINE3	EBX = 0000001E Yval = 0000001E	Move the value of Yval into register EBX
LINE4	EBX = FFFFFFFF6 Zval = 00000028	Value of register EBX subtract with value of Zval and store in EBX
LINE5	EAX = FFFFFFFDC EBX = FFFFFFFF6	Add value of register EAX and EBX and store in EAX
LINE6	EAX = FFFFFFFDD	Increase the value of register EAX by 1
LINE7	EAX = FFFFFFFDD Rval = FFFFFFFDD	Move the value from register EAX to Rval

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

```
EAX = 0000001A EBX = 003B8000 ECX = 00761005 EDX = 00761005 ESI = 00761005 EDI = 00761005
EIP = 00761015 ESP = 004FFD04 EBP = 004FFD10 EFL = 00000246
```

LINE2:

```
EAX = FFFFFFFE6 EBX = 003B8000 ECX = 00761005 EDX = 00761005 ESI = 00761005 EDI = 00761005  
EIP = 00761017 ESP = 004FFD04 EBP = 004FFD10 EFL = 00000293  
  
0x00764008 = 0000001E
```

LINE3:

```
EAX = FFFFFFFE6 EBX = 0000001E ECX = 00761005 EDX = 00761005 ESI = 00761005 EDI = 00761005  
EIP = 0076101D ESP = 004FFD04 EBP = 004FFD10 EFL = 00000293  
  
0x0076400C = 00000028
```

LINE4:

```
EAX = FFFFFFFE6 EBX = FFFFFFFF6 ECX = 00761005 EDX = 00761005 ESI = 00761005 EDI = 00761005  
EIP = 00761023 ESP = 004FFD04 EBP = 004FFD10 EFL = 00000287
```

LINE5:

```
EAX = FFFFFFFDC EBX = FFFFFFFF6 ECX = 00761005 EDX = 00761005 ESI = 00761005 EDI = 00761005  
EIP = 00761025 ESP = 004FFD04 EBP = 004FFD10 EFL = 00000283
```

LINE6:

```
EAX = FFFFFFFDD EBX = FFFFFFFF6 ECX = 00761005 EDX = 00761005 ESI = 00761005 EDI = 00761005  
EIP = 00761026 ESP = 004FFD04 EBP = 004FFD10 EFL = 00000287  
  
0x00764000 = 00000000
```

LINE7:

```
EAX = FFFFFFFDD EBX = FFFFFFFF6 ECX = 00761005 EDX = 00761005 ESI = 00761005 EDI = 00761005  
EIP = 0076102B ESP = 004FFD04 EBP = 004FFD10 EFL = 00000287
```

Q3. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 3 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = [(\text{var1} * \text{var2}) + \text{var3}] - 1$

```
include irvine32.inc

.data
var1 DWORD 5
var2 DWORD 10
var3 DWORD 20
var4 DWORD ?

.code
main proc
    mov eax, var1        ; LINE1
    mul var2              ; LINE2
    add eax, var3         ; LINE3
    dec eax               ; LINE4
    exit
main endp
end main
```

Answer Q3

a) Fill (Write) in the contents for the related register in each line:

Table 3

LINE1	EAX = 00000005h var1 = 00000005h	Move the value of var1 (5d) into register EAX
LINE2	EAX = 00000032h var2 = 0000000Ah	Multiply the value store in register EAX by the value from var2 (10d). Then store the result in EAX.
LINE3	EAX = 00000046h var3 = 00000014h	Add value register of EAX and value of var3 (20d) and store in EAX.
LINE4	EAX = 00000045h var4 = 0	Decrease the value of EAX by 1

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

```
EAX = 00000005 EBX = 00485000 ECX = 00281005 EDX = 00281005 ESI = 00281005 EDI = 00281005
EIP = 00281015 ESP = 0038F7F0 EBP = 0038F7FC EFL = 00000246

0x00284004 = 0000000A
```

LINE2:

```
EAX = 00000032 EBX = 00485000 ECX = 00281005 EDX = 00000000 ESI = 00281005 EDI = 00281005
EIP = 0028101B ESP = 0038F7F0 EBP = 0038F7FC EFL = 00000202

0x00284008 = 00000014
```

LINE3:

```
EAX = 00000046 EBX = 00485000 ECX = 00281005 EDX = 00000000 ESI = 00281005 EDI = 00281005  
EIP = 00281021 ESP = 0038F7F0 EBP = 0038F7FC EFL = 00000202
```

LINE4:

```
EAX = 00000045 EBX = 00485000 ECX = 00281005 EDX = 00000000 ESI = 00281005 EDI = 00281005  
EIP = 00281022 ESP = 0038F7F0 EBP = 0038F7FC EFL = 00000202
```

Q4. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 4 with the content of each register or variable on every LINE, in Hexadecimal (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = (\text{var1} * 5) / (\text{var2} - 3)$

```
include irvine32.inc
.data
    var1 WORD 40
    var2 WORD 10
    var4 WORD ?
.code
main proc
    mov ax,var1      ; LINE1
    mov bx,5         ; LINE2
    mul bx           ; LINE3
    mov bx,var2      ; LINE4
    sub bx,3         ; LINE5
    div bx           ; LINE6
    mov var4,ax      ; LINE7
    exit
main endp
end main
```

Answer Q4

a) Fill (Write) in the contents for the related register in each line:

Table 4

LINE1	AX = 0028h var1 = 0028h	Move the value of var1 (40d) into register AX
LINE2	BX = 0005	Move the value of 5 into register BX
LINE3	AX = 00C8 BX = 0005	Multiply the value store in register AX by the value from register BX. Then store the result in AX
LINE4	BX = 000A var2 = 000A	Move the value of var2 which is 10d into register BX
LINE5	BX = 0007	Subtract the value stored in register BX with a value of 3. Then store result in BX
LINE6	AX = 001C BX = 0007 DX = 0004	Divide the value stored in register AX by the value from register BX. Then the quotient is stored in AX while the remainder is stored in DX
LINE7	AX = 001C var4 = 001C	Move the value of register AX into var4

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

```
EAX = 00F50028 EBX = 011D6000 ECX = 00E51005 EDX = 00E51005 ESI = 00E51005 EDI = 00E51005
EIP = 00E51016 ESP = 00F5FDD8 EBP = 00F5FDE4 EFL = 00000246
```

LINE2:

```
EAX = 00F50028 EBX = 011D0005 ECX = 00E51005 EDX = 00E51005 ESI = 00E51005 EDI = 00E51005  
EIP = 00E5101A ESP = 00F5FDD8 EBP = 00F5FDE4 EFL = 00000246
```

LINE3:

```
EAX = 00F500C8 EBX = 011D0005 ECX = 00E51005 EDX = 00E50000 ESI = 00E51005 EDI = 00E51005  
EIP = 00E5101D ESP = 00F5FDD8 EBP = 00F5FDE4 EFL = 00000202
```

LINE4:

```
EAX = 00F500C8 EBX = 011D000A ECX = 00E51005 EDX = 00E50000 ESI = 00E51005 EDI = 00E51005  
EIP = 00E51024 ESP = 00F5FDD8 EBP = 00F5FDE4 EFL = 00000202
```

LINE5:

```
EAX = 00F500C8 EBX = 011D0007 ECX = 00E51005 EDX = 00E50000 ESI = 00E51005 EDI = 00E51005  
EIP = 00E51028 ESP = 00F5FDD8 EBP = 00F5FDE4 EFL = 00000202
```

LINE6:

```
EAX = 00F5001C EBX = 011D0007 ECX = 00E51005 EDX = 00E50004 ESI = 00E51005 EDI = 00E51005  
EIP = 00E5102B ESP = 00F5FDD8 EBP = 00F5FDE4 EFL = 00000202
```

LINE7:

```
EAX = 00F5001C EBX = 011D0007 ECX = 00E51005 EDX = 00E50004 ESI = 00E51005 EDI = 00E51005  
EIP = 00E51031 ESP = 00F5FDD8 EBP = 00F5FDE4 EFL = 00000202
```

Short Notes for MUL CX and DIV BL:

MUL CX

- a. MUL always uses AX (or its extended versions EAX or RAX) as the implicit destination register.
- b. The operand size determines the size of the result:
 - i. Byte-sized operand: Result in AX
 - ii. Word-sized operand: Result in DX:AX
 - iii. Doubleword-sized operand (32-bit mode): Result in EDX:EAX
 - iv. Quadword-sized operand (64-bit mode): Result in RDX:RAX
- c. The upper half of the result (DX or EDX or RDX) holds any overflow bits.
- d. The Carry Flag (CF) is set if the upper half of the product is non-zero.

DIV BL

- a. DIV always uses the DX:AX or EDX:EAX pair as the implicit dividend register.
- b. The divisor is specified as the operand of the DIV instruction.
- c. The quotient is stored in AX (for 16-bit division) or EAX (for 32-bit division).
- d. The remainder is stored in DX.
- e. Clear DX (or EDX for 32-bit division) before division to ensure a correct 16-bit or 32-bit dividend.
- f. If the divisor is 0, a division error occurs.

- g. The Overflow Flag (OF) is set if the quotient is too large to fit in the destination register.

Q5. Given the following instructions as is Code Snippet 1.

- a) Write a full program to execute the Code Snippet 1.
b) What are the contents of the related registers after Code Snippet 1 is executed? Paste the screenshot of DumpReg.

; Code Snippet 1 (MUL CX)

```
MOV DX, 0      ; Clear DX
MOV AX, 1000h   ; Load 1000h into AX
MOV CX, 25h     ; Load 25h into CX
MUL CX         ; Multiply AX by CX, storing the result in DX:AX
```

Answer Q5

- a) Screenshot of full program (.asm) :

```
1  include Irvine32.inc
2
3  .code
4  main proc
5      ; Code Snippet 1 (MUL CX)
6
7      MOV DX, 0; Clear DX
8      MOV AX, 1000h; Load 1000h into AX
9      MOV CX, 25h; Load 25h into CX
10     MUL CX; Multiply AX by CX, storing the result in DX : AX
11     call DumpRegs
12
13     exit
14     main endp
15     end main
```

- b) Paste here the screenshot of the final registers' content (DumpReg):

```
EAX=00D95000  EBX=00F1C000  ECX=009A0025  EDX=009A0002
ESI=009A10AA  EDI=009A10AA  EBP=00D9F980  ESP=00D9F974
EIP=009A3674  EFL=00000A07  CF=1  SF=0  ZF=0  OF=1  AF=0  PF=1
```

Q6. Given the following instructions as is Code Snippet 2.

- a) Write a full program to execute the Code Snippet 2.
- b) What are the contents of the related registers after Code Snippet 2 is executed? Paste the screenshot of DumpReg.

; Code Snippet 2 (DIV BL)

```
MOV DX, 0      ; Clear DX to form the 16-bit dividend in DX:AX
MOV AX, 803h   ; Load the dividend (8003h) into AX
MOV BL, 10h    ; Load the divisor (10h) into BL
DIV BL         ; Divide DX:AX by BL, whereby AX=quotient & DX=remainder
```

Answer Q6

- a) Screenshot of full program (.asm) :

```
1  include irvine32.inc
2
3  .code
4  main proc
5      ; Code Snippet 2 (DIV BL)
6      MOV DX, 0; Clear DX to form the 16 - bit dividend in DX : AX
7      MOV AX, 803h; Load the dividend(803h) into AX
8      MOV BX, 10h; Load the divisor(10h) into BL
9      DIV BX; Divide DX : AX by BL, whereby AX = quotient & DX = remainder
10     call DumpRegs    ≤ 1ms elapsed
11
12     exit
13 main endp
14 end main
```

- b) Paste here the screenshot of the final registers' content (DumpReg):

```
EAX=006F0080  EBX=00500010  ECX=000F10AA  EDX=000F0003
ESI=000F10AA  EDI=000F10AA  EBP=006FFB1C  ESP=006FFB10
EIP=000F3674  EFL=00000246  CF=0  SF=0  ZF=1  OF=0  AF=0  PF=1
```